

Rapport MASB TP3

Présenté par :

LEMONNIER Paul
CHAUSSIEZ Benoit

Master 1: ***Méthodes informatiques et
statistiques pour les omics (MISO)***

Année universitaire : 2025-2026

Table des Matières

1) Introduction	3
2) Dictionnaire (Paul)	3
2.1) Le Moteur (Structure de Données)	3
2.2) La Transmission (Filtrage et Nettoyage)	4
2.3) La Carrosserie (Paramètres utilisés)	4
2.3.1) Paramètres Généraux	4
2.4) Pneu (Extension)	4
2.4.1) Pneu droit (Extension Droite)	4
2.4.2) Pneu Gauche (Extension Gauche)	4
2.5) Joint de culasse (Résolution des complications)	5
2.6) Résultats de Performance (Résultat expérimental)	5
2.7) Étude détaillée par niveau d'assemblage	6
Niveau 0 : Débugage	6
Niveau 1 et 2 : Premiers défis	7
Niveau 3 : De plus en plus complexe	9
Niveau 4 : Introduction d'hétérozygotie (Ploïdie)	11
Niveau 5 : Répétitions complexes	14
Niveau 6 : Moins de répétitions	15
2.8) Voies d'amélioration	16
3) Filtre de Bloom (Benoît)	16
3.1) Le Moteur (Bloom Filter)	16
3.2) La Transmission (Réduction des k-mers)	17
3.3) La Carrosserie (Paramètres utilisés)	18
3.3.1) Paramètres Généraux	18
3.3.2) Calcul de la taille optimale du Bloom Filter et du nombre de fonction de hachage	18
3.4) Pneu (Extension)	19
3.4.1) Pneu droit(Extension Droite)	19
3.4.2) Pneu Gauche (Extension Gauche)	20
3.5) Joint de culasse (Fusion de contigs)	20
3.5.1) Multi seeds.	20
3.5.2) Fusion avec multi seeds	21
3.6) L'influence du multi seed (Analyse Comparative)	21
3.7) Gestion de dodane(Bulle)	21
3.8) Voie d'amélioration	22
3.8.1)Gestion optimisée des faux positifs	22
3.9) Contrôle technique (Quast)	22
3.10) Résultat de Performance (Résultat expérimental)	23
3.10.1) Resultat de temps et de taille de génome	23
3.10.2) Résultat de KAT sur les niveaux 4 5 6	24
4) Conclusion Comparative	28

1) Introduction

Le séquençage à haut débit produit aujourd'hui des quantités massives de données, mais ces lectures (reads) sont trop courtes pour être exploitables directement. L'assemblage de génomes consiste à reconstruire la séquence d'ADN originale à partir de ces fragments, un problème fondamental en bioinformatique.

Ce projet vise à implémenter un assembleur basé sur le graphe de Bruijn, une structure de données élégante qui représente les chevauchements entre les k-mers (sous-chaînes de longueur k). L'objectif est triple Implémenter un assembleur fonctionnel capable de reconstruire des génomes à partir de données de séquençage. Assembler plusieurs jeux de données présentant des difficultés croissantes (niveaux 0 à 10). Évaluer la qualité des assemblages à l'aide d'outils standards comme QUAST.

Le TP se base sur un ensemble de niveau de 0 à 10

Ce rapport détaille l'implémentation, les résultats obtenus pour chaque niveau, et une discussion des limites de l'approche.

2) Dictionnaire (Paul)

L'approche par dictionnaire repose sur le stockage explicite de chaque k-mer unique rencontré dans le jeu de données, associé à son occurrence (abondance). Contrairement au filtre de Bloom, cette méthode permet une gestion précise des seuils de confiance et une exploration exhaustive du graphe de De Bruijn.

2.1) Le Moteur (Structure de Données)

Le dictionnaire (hash table) stocke les k-mers comme clés et leur fréquence comme valeurs. Cette structure garantit une complexité de recherche en $O(1)$, essentielle pour les phases d'extension de contigs. Le dictionnaire est construit après un passage unique sur les lectures, suivi d'un filtrage des k-mers à faible occurrence pour éliminer les erreurs de séquençage.

2.2) La Transmission (Filtrage et Nettoyage)

Le filtrage est basé sur le spectre des k-mers. Le paramètre *t* (seuil) permet de distinguer le bruit (erreurs de séquençage) du signal biologique.

- **Paramètre -s (Target Size)** : Définit la quantité totale de k-mers à assembler. Il est crucial pour arrêter l'algorithme une fois le génome couvert, évitant ainsi d'assembler du bruit ou des artefacts.
- **Paramètre -m (Min Contig Size)** : Filtre les contigs trop courts en fin de processus. Il permet d'épurer le rapport final des fragments non significatifs issus de zones de très faible couverture ou d'erreurs.

2.3) La Carrosserie (Paramètres utilisés)

2.3.1) Paramètres Généraux

L'équilibre de l'assemblage repose sur :

- **k** : Longueur du k-mer. Un *k* élevé résout les répétitions ; un *k* faible améliore la connectivité.
- **t** : Seuil d'occurrence des kmers. des kmers présents qu'une seule fois sont parfois à supprimer pour alléger l'assemblage
- **d** : Profondeur du *lookahead* pour la résolution de conflits. Cela permet d'avoir une vision plus ou moins loin lors d'un conflit afin d'analyser les chemins possibles : si un chemin s'arrête, on va préférer en prendre un autre qui continue.

2.4) Pneu (Extension)

2.4.1) Pneu droit (Extension Droite)

Recherche du successeur (A, C, G, T) le plus abondant dans le dictionnaire. En cas de bifurcation, l'algorithme anticipe sur *d* bases pour choisir le chemin le plus probable.

2.4.2) Pneu Gauche (Extension Gauche)

Extension symétrique utilisant le complément inverse pour reconstruire le contig dans la direction opposée à partir de la *seed*.

2.5) Joint de culasse (Résolution des complications)

- **Hétérozygotie** : Résolue par le *lookahead* qui permet de choisir un chemin consensus à travers les bulles.
- **Répétitions** : Traversées en augmentant la valeur de k au-delà de la taille de la répétition et en modifiant le paramètre s; le nombre de kmers à assembler
- **Faible couverture** : Gérée en abaissant t et k pour maintenir la continuité.

2.6) Résultats de Performance (Résultat expérimental)

Niveau	Paramètres (k,t,d,s,m)	Nmax (pb)	N50 (pb)	Contigs	Temp en secondes	Nombre d'essais
L0	21, 1, 50, 1.5M, 9990	9991	9991	1	1.34 secondes	1
L1	55, 3, 50, 1.3M, 800k	838178	838178	1	18.76 secondes	5
L2	31, 3, 50, 1.5M, 5k	999976	999976	1	18.40 secondes	3
L3	31, 3, 50, 1.2M, 900k	958079	958079	1	23.45 secondes	3

L4	31, 3, 50, 3M, 800k	8823 15	171 983 1	2	36.66 secondes	4
L5	71, 8, 150, 2.5M, 10k	426 687	413 659	10	20.98 secondes	8
L6	31, 6, 50, 2M, 15k	4368 8	263 21	67	29.04 secondes	20

Tableau 2.1 – Performances de l'assemblage pour les niveaux 0 à 6. Paramètres : *k* = Taille des kmers, *t* = seuil d'abondances des kmers utilisées pour l'assemblage, *d* = Profondeur du lookahead pour la résolution de conflits, *s* = quantité totale autorisée de *k*-mers à assembler, *m* = Taille minimum des contigs gardés dans le fichier d'assemblage. *Nmax* = taille du plus grand contig, *N50* = Taille du contig représentant la moitié de l'assemblage, *Temps* = le temps en seconde depuis l'ouverture du fichier de reads jusqu'à la génération du fichier FASTA contenant l'assemblage.

2.7) Étude détaillée par niveau d'assemblage

Cette section analyse les défis spécifiques rencontrés lors de la progression des niveaux et justifie les choix paramétriques à travers les outils QAST et KAT.

Niveau 0 : Débugage

Ce niveau est sans aucune erreur et sert à vérifier si l'algorithme d'assemblage fonctionne correctement.

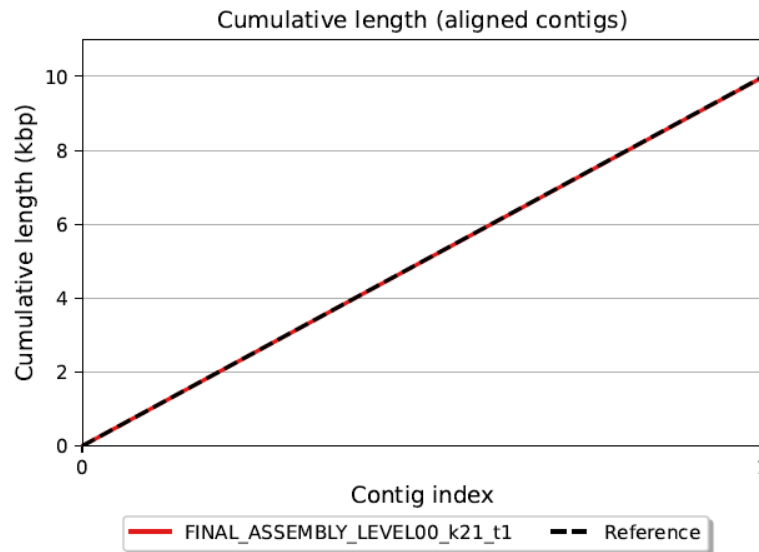


Figure 2.1 : Courbe de la longueur cumulée (*Cumulative length*) du contig assemblé par rapport au génome de référence du niveau 0.

On voit bien que sur la figure 2.1, la courbe du contig assemblé est bien superposée à celle du génome de référence, ce qui signifie que le contig correspond bien à la majorité du génome de référence.

- **Validation QUAST :** On observe un N50 de 9991 égal à la longueur du contig, une *Genome Fraction* de 99.910% et aucun morceau génome non aligné, ce qui confirme la bonne reconstruction du génome

Niveau 1 et 2 : Premiers défis

Ces deux premiers niveaux sont là pour commencer à introduire des complications et à étudier comment le changement de la taille des kmers k et du filtrage d'abondance des kmers t peut améliorer l'alignement.

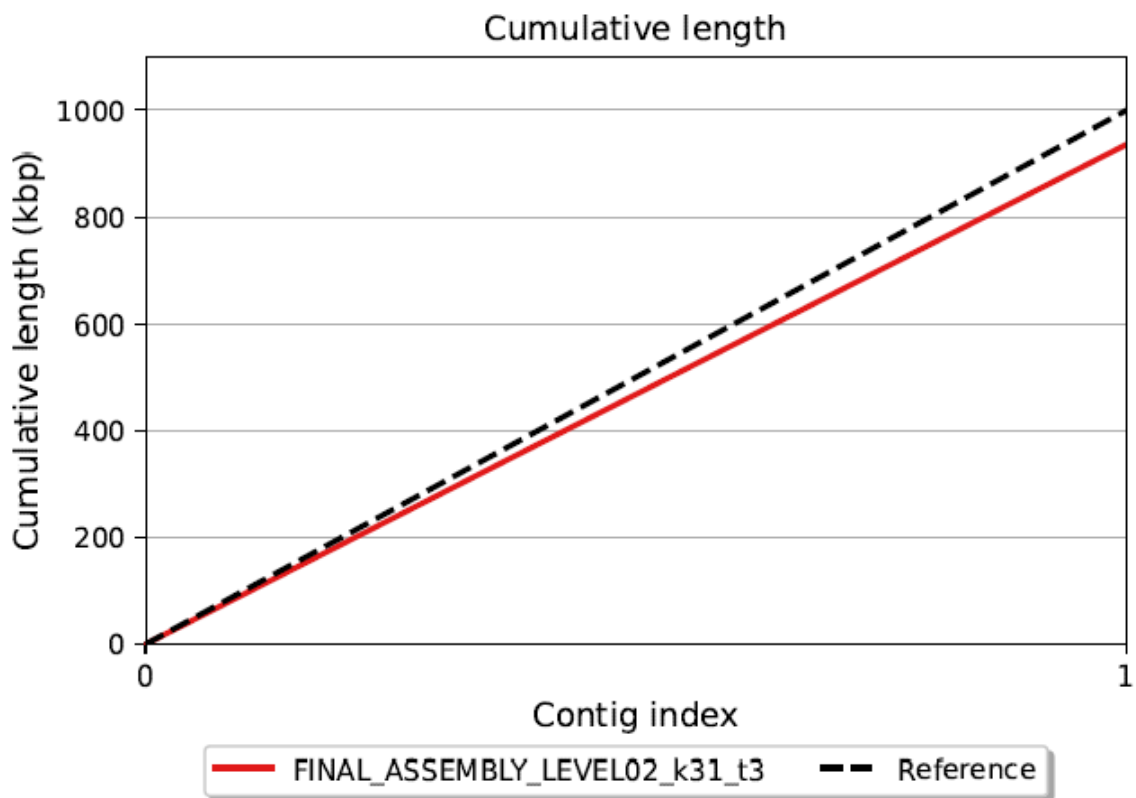
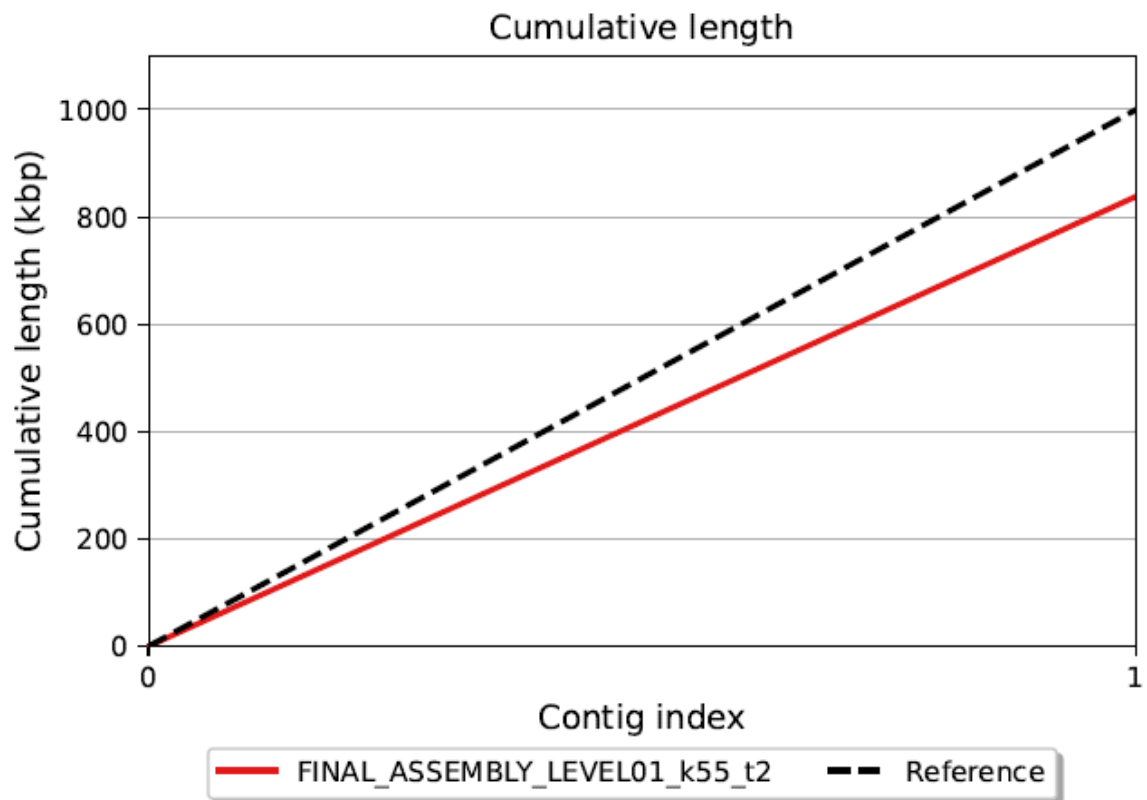


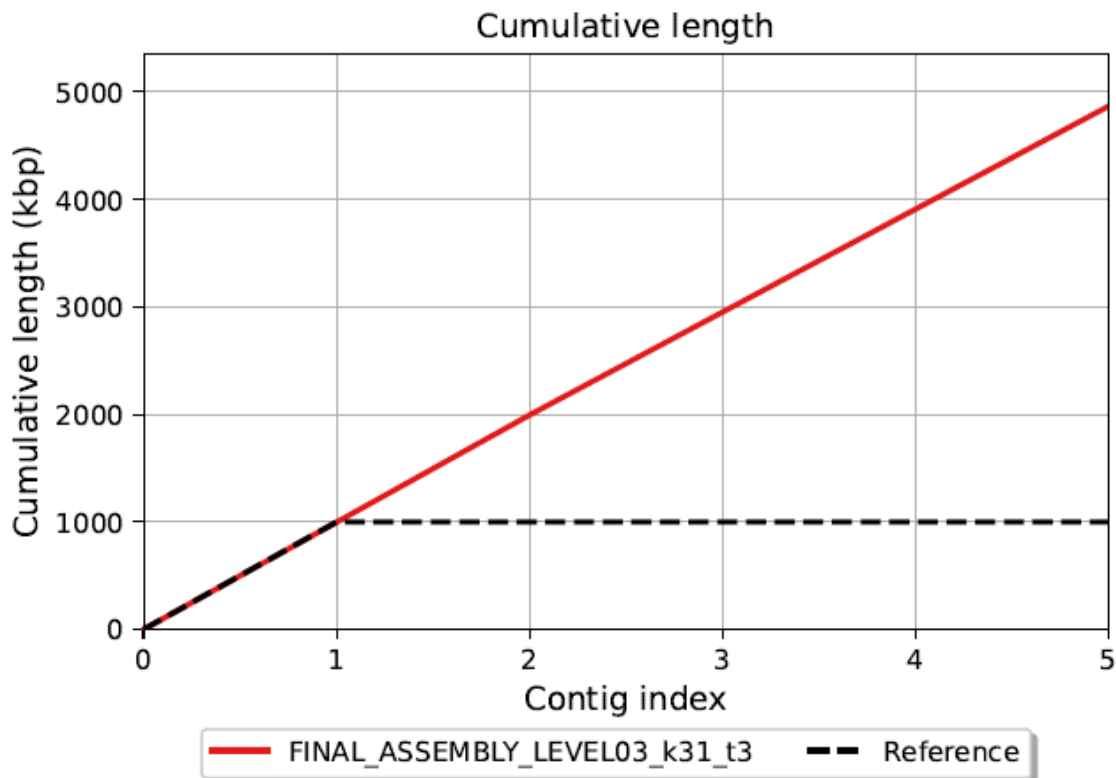
Figure 2.2 et 2.3 : Courbes de la longueur cumulée (Cumulative length) du contig assemblé par rapport au génome de référence du niveau 1 et 2.

- **Validation QUAST :**

- Niveau 1 : On observe un N50 de 838178, égal à la longueur du contig, une *Genome Fraction* de 83.818% et aucun morceau génome non aligné.
- Niveau 2 : On observe un N50 de 935961, égal à la longueur du contig, une *Genome Fraction* de 93.596% et aucun morceau génome non aligné.
- On peut voir que ce soit dans le rapport QUAST et sur les deux graphiques (figures 2.2 et 2.3) que la longueur du génome est de 1 million de bp, ce qui montre que l'on a pu reconstruire la grande majorité du génome.

Niveau 3 : De plus en plus complexe

Le niveau 3 semble introduire des répétitions de k-mers qui commencent à tester la robustesse de la taille de k ainsi que la précision de s qui permet de supprimer ses répétitions



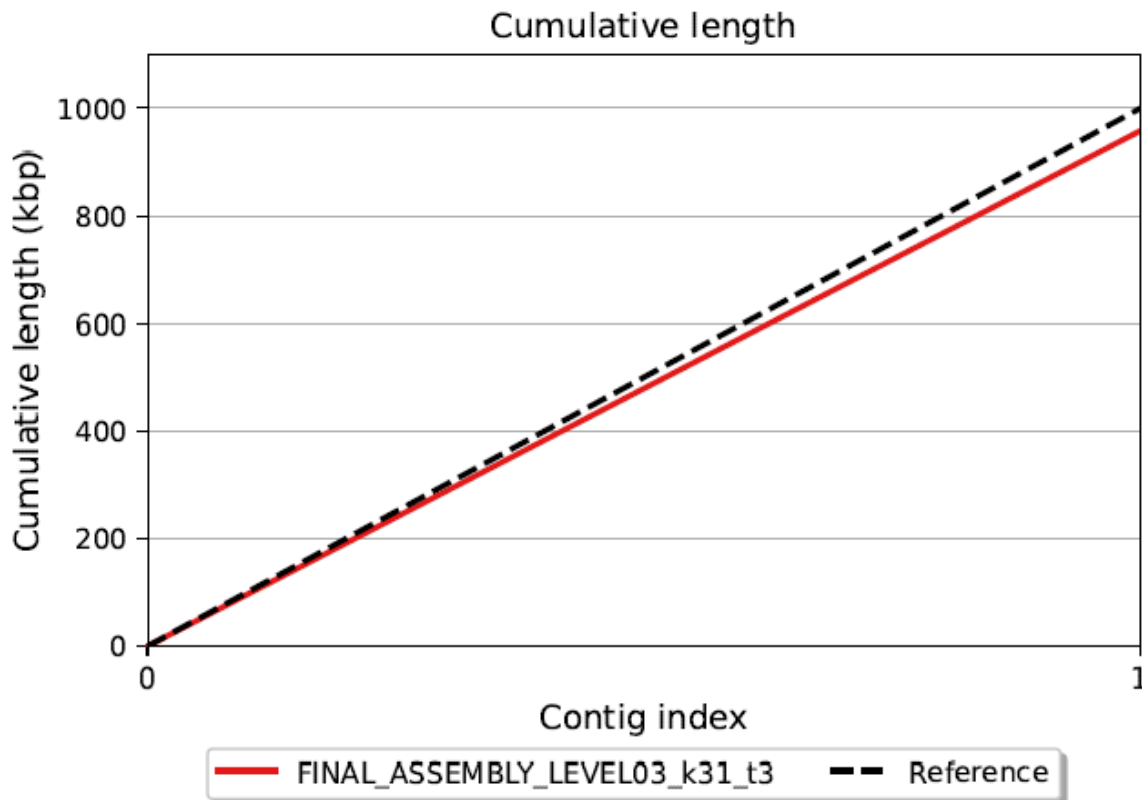


Figure 2.4 et 2.5 : Courbes de la longueur cumulée (*Cumulative length*) du contig assemblé par rapport au génome de référence du niveau 3; la figure 4 est produite avec un assemblage maximum de 5Mpb et la figure 5, avec un assemblage maximum de 1.2 Mpb.

Ces deux graphiques (figures 2.4 et 2.5) montrent l'importance du paramètre *s*; la taille maximale de k-mers assemblés, autrement dit la taille finale du fichier fasta. En passant ce paramètre à millions de pb, on permet d'aligner plus de k-mers mais on voit bien que sur la figure 4, la taille réelle du génome de référence est de 1M pb alors que notre assembleur est de 5M pb. Il semble avoir assemblé beaucoup plus que nécessaire. Le *Duplication ratio* de 4.881, montre que l'assembleur a produit 5 fois la même séquence. Pour un *s* de 1.2 Mpb, on obtient toujours une *Genome fraction* de 99.752% mais cette fois-ci sans duplication (figure 2.5)

- **Validation QUAST :** On observe un N50 égal de 958079 à la longueur du contig, une *Genome Fraction* de 99.752% et aucun morceau génome non aligné, ce qui confirme la bonne reconstruction du génome.

Niveau 4 : Introduction d'hétérozygotie (Ploïdie)

Ce niveau semble simuler un organisme diploïde où les deux allèles divergent par des mutations ponctuelles, créant des "bulles" dans le graphe. En effet, lorsque l'on regarde le graphique des kmers (figure 2.6) qui permet d'étudier l'abondance des kmers distincts présents dans le dictionnaire formé depuis les reads, on voit deux courbes en cloches, une autour de 40X et une autre autour de 80X, ce qui semble montrer que les kmers de 40X sont lu deux fois moins que ceux à 80X. Cela démontre que certains kmers sont présents deux fois moins que d'autres, ce qui va introduire une bulle dans notre graph de Bruijn et peut potentiellement signifier une hétérozygotie, comme par exemple un allèle avec une mutation sur un des deux chromosomes.

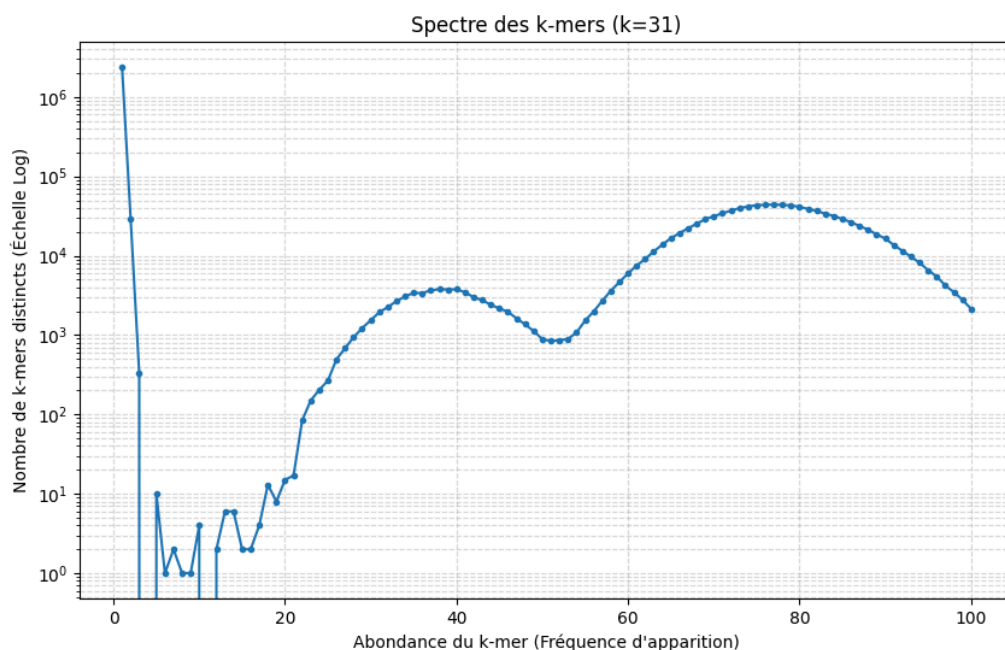


Figure 2.6 : Courbe de l'abondance des kmers distinct du dictionnaire généré par les reads du niveau 4. L'axe horizontal indique la multiplicité (le nombre de fois qu'un k-mer apparaît dans le jeu de données). L'axe vertical indique le nombre de k-mers uniques possédant cette multiplicité précise. Un spectre typique se décompose en deux parties distinctes : la pente des erreurs (à gauche, zone 1x à ~5x) : On observe un pic très élevé près de l'origine. Il s'agit des k-mers créés par des erreurs de séquençage. Comme les erreurs sont aléatoires, ces k-mers n'apparaissent généralement qu'une seule fois. La ou les courbes en cloches (le pic principal) : Plus

loin sur l'axe X, on observe une ou des cloches qui représentent le signal réel du génome. La position du sommet de cette colline indique la couverture moyenne du séquençage (par exemple 40x ou 80x). Ce graphique peut être utile pour statuer sur la valeur du paramètre t (le seuil de sélection des kmers selon leur abondance) qui va se situer entre la pente d'erreurs et le début de la cloche.

Il était important de réussir à récupérer, grâce aux différents essais, la taille maximale des contigs pour ne garder que les plus importants qui peuvent contenir le génome de référence et filtrer les branches secondaires (Tip removal), ainsi que de laisser une taille maximale de kmers suffisamment grande ($s = 3\text{M pb}$) pour pouvoir explorer les deux chemins possibles introduits par la bulle.

- **Validation KAT :** (figure 2.7) Le spectre affiche plusieurs pics tous autour de $80X = \text{kmers présents } 80 \text{ fois dans les reads mais pas forcément dans l'assemblage}$:
 - Un pic noir (0x): certains kmers ne sont pas retrouvés dans l'assemblage final
 - Un pic rouge (1x): Les kmers qui ne sont présents qu'une seule fois dans l'assemblage final
 - Un pic violet (2x): indique que la majorité des kmers sont présents deux fois dans l'assemblage, probablement une fois par allèles

Le fait d'obtenir dans l'assemblage final deux contigs, qui semblent très similaires (sur le KAT, la majorité des kmers sont présents deux fois, donc un par contigs, donc très probablement identiques) peut donc signifier que l'on retrouve les deux versions de l'allèle. A noter que cela peut aussi être le brin complémentaire (moins probable du fait de certains kmers présents qu'une seule fois).

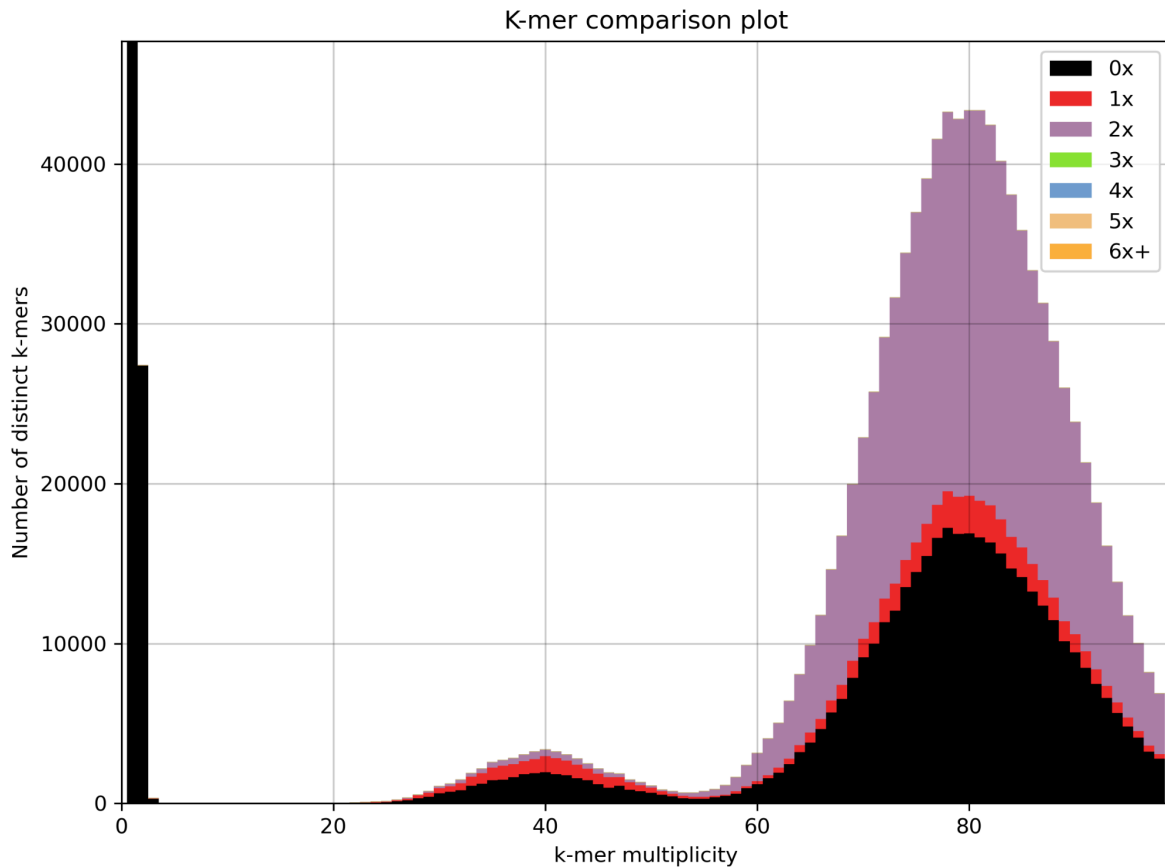


Figure 2.7 : Courbe de l'abondance des kmers distinct présent dans l'assemblage pour le niveau 4. Le programme KAT (Kmer Analysis Toolkit) génère tous les kmers possible depuis le fichier de reads et analyse s'ils sont présents dans l'assemblage produit par l'assembleur. L'axe horizontal indique la multiplicité ou l'abondance (le nombre de fois qu'un k-mer apparaît dans les reads). L'axe vertical indique le nombre de k-mers uniques possédant cette multiplicité précise. Les couleurs indiquent l'absence ou la présence (une fois ou plus) des kmers obtenus depuis les reads au sein de l'assemblage. Noir (0x) = kmers présents dans les reads mais absents de l'assemblage; rouge (1x) = kmers présents dans les reads et présents une fois dans l'assemblage; violet (2x) = kmers présents dans les reads et présents deux fois dans l'assemblage; vert (3x) = kmers présents dans les reads et présents trois fois dans l'assemblage; bleu (4x) = kmers présents dans les reads et présents quatre fois dans l'assemblage; ; jaune clair/beige(5x) = kmers présents dans les reads et présents cinq fois dans l'assemblage; jaune/orange (6x+) = kmers présents dans les reads et présents six fois ou plus dans l'assemblage.

Niveau 5 : Répétitions complexes

Le niveau 5 semble caractérisé par des répétitions plus longues que pour une valeur de k classique (31) ce qui va introduire plein de redondance dans le graph de de Bruijn. Il a donc fallu modifier tous les paramètres; augmenter la taille des kmers à 71, pour essayer de passer outre ces répétitions, le seuille d'abondance à 8 pour être plus strict au niveau des erreurs de séquençage, la distance d'analyse des conflits à 150 pour regarder plus loin afin de s'assurer que l'on prends un chemin valide, une taille totale de kmers à 2.5 millions pour en capturer le plus possible et une taille de kmers individuels à 10 milles car les plus faibles ne seront pas informatifs.

- **Validation KAT :** (figure 2.8) On observe une superposition des courbes (1x à 6x+) sur le pic principal. On voit qu'il y a une majorité de kmers absents et présent une seule fois (couleurs noir et rouge plus présentes) et une minorité de kmers présents 2-3-4-5 fois (couleurs violet, vert, bleu et beige moins présentes). On a donc réussi à capturer le squelette principal du génome dans l'assemblage mais il reste encore des séquences répétées et donc avec de la redondance

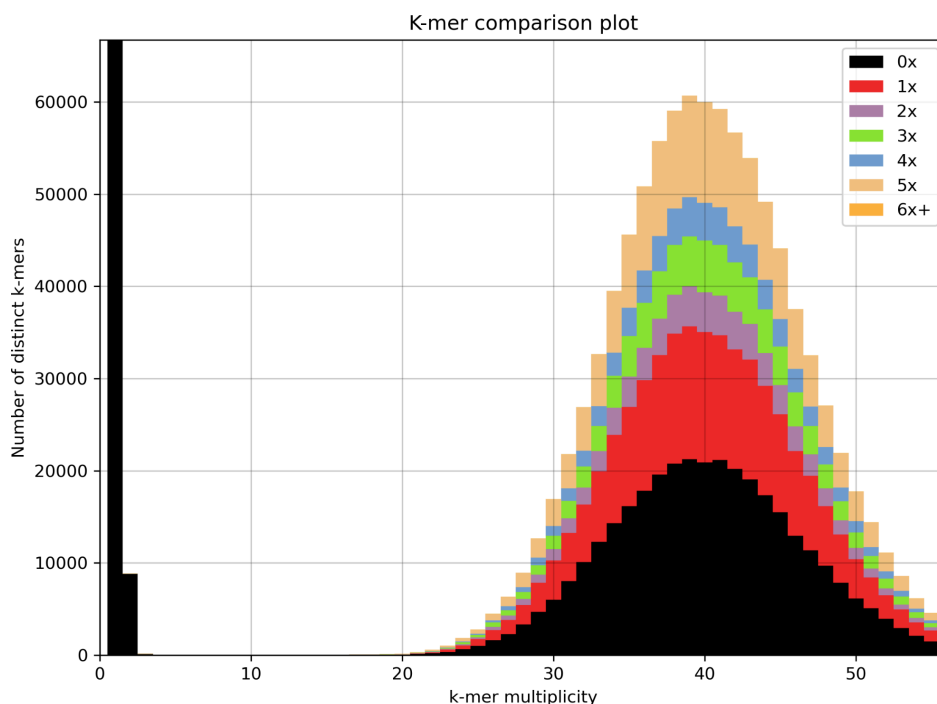


Figure 2.8 : Courbe de l'abondance des kmers distinct présent dans l'assemblage pour le niveau 5. Indications sur la légende dans la figure 6.

Niveau 6 : Moins de répétitions

Pour ce dernier niveau, après de nombreux essais, j'ai réussi à obtenir un assemblage qui semble avoir reconstruit une bonne partie du génome.

- **Validation KAT :** Le spectre (figure 2.9) montre un pic rouge (1x) entre 30x et 35x, majoritaire, avec quand même la présence d'un pic noir (0x), reflétant un assembleur incomplet et un léger pic violet (2x) montrant encore la présence de quelques duplications.

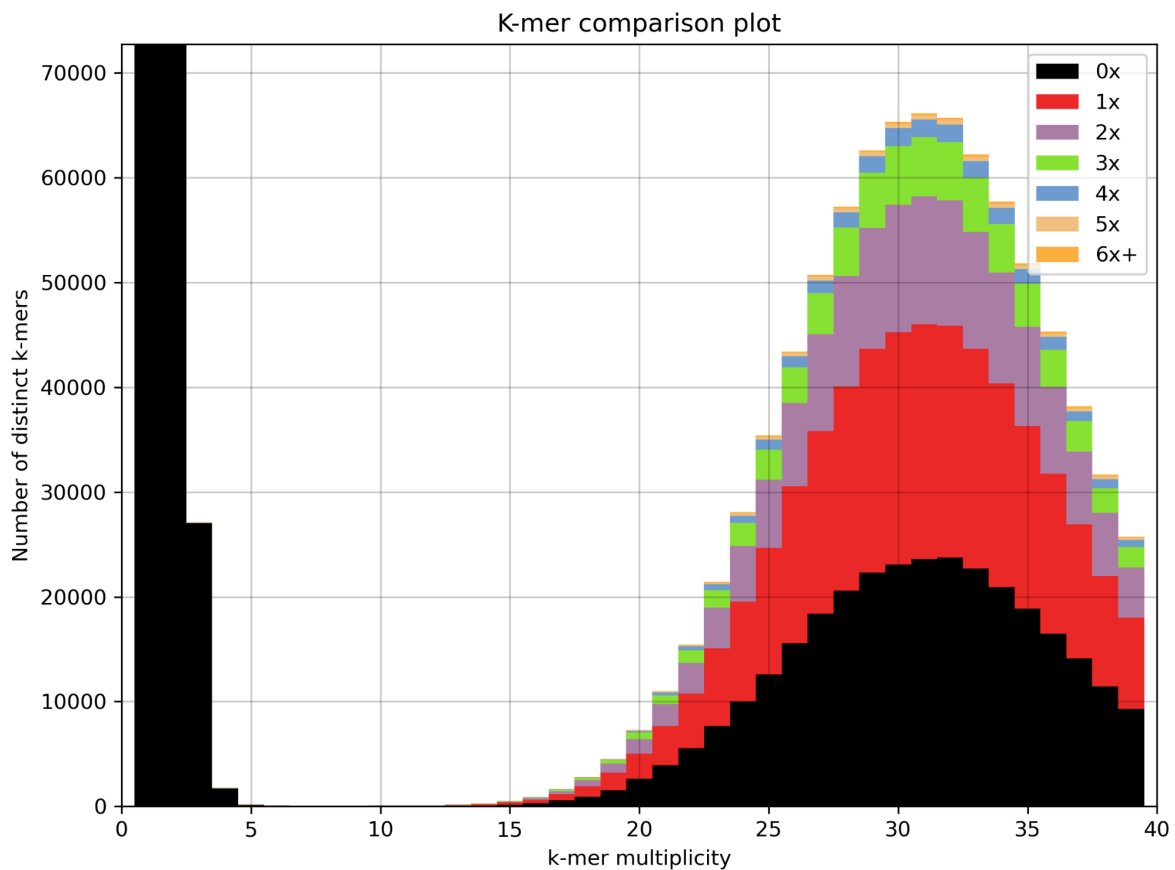


Figure 2.9 : Courbe de l'abondance des kmers distinct présent dans l'assemblage pour le niveau 6. Indications sur la légende dans la figure 6

2.8) Voies d'amélioration

Bien que l'assembleur par dictionnaire ait prouvé sa robustesse jusqu'au Niveau 6, plusieurs optimisations techniques pourraient être envisagées pour franchir les limites de l'assemblage :

- Gestion des répétitions : Pour résoudre la fragmentation observée au Niveau 5, l'intégration de l'information de distance (lectures appariées ou *paired-ends*) permettrait de relier les contigs entre eux en sautant par-dessus les zones répétées ou les gaps de couverture.
- Correction d'erreurs pré-assemblage : L'ajout d'un module de correction basé sur la multiplicité des k-mers permettrait de lisser la grosse pente d'erreurs observée sur les spectres, augmentant ainsi la fiabilité des k-mers uniques rencontrés au Niveau 6.

3) Filtre de Bloom (Benoit)

3.1) Le Moteur (Bloom Filter)

Le filtre de Bloom est une structure de données probabiliste permettant de représenter un ensemble d'éléments (comme un k-mer, une molécule⁽¹⁾ ou un génome) sous la forme d'un tableau de bits à l'aide de fonctions de hachage. Il permet ensuite de déterminer si un autre élément appartient probablement à l'ensemble des éléments déjà insérés dans ce tableau.

L'application du Filtre de Bloom se base sur 2 étapes , il y a l'élaboration du filtre de bloom par l'insertion de l'élément qui nous intéresse. Cette élément est inséré dans un tableau de bits par des fonctions de hachage qui vont renvoyer des positions dans le tableau de bits où un 1 sera indiqué .Cette combinaison de 1 représente l'élément dans le tableau , il faut effectuer cette opération avec plusieurs éléments qui vont s'intégrer dans le tableau pour avoir l'ensemble d'éléments.

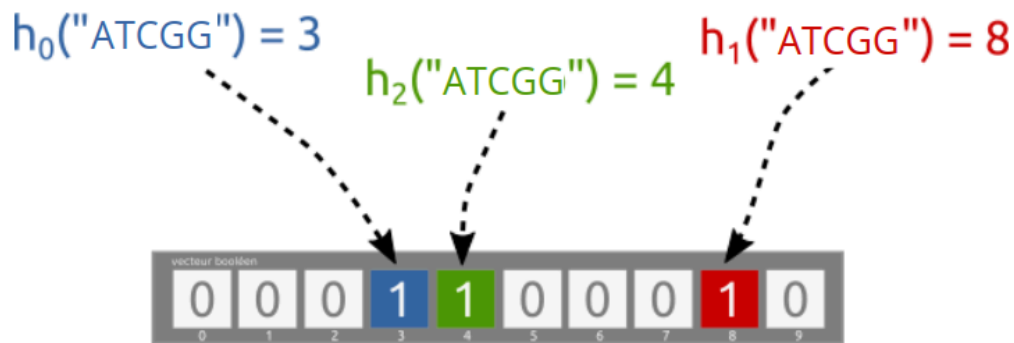


Figure 3.1 : Illustration de l'encodage d'une graine (*k*-mer) dans un filtre de Bloom avec $k=3$ fonctions de hachage.

Lors de l'insertion d'un *k*-mer de taille 5 dans un filtre de Bloom, celui-ci est traité par trois fonctions de hachage distinctes. Chaque fonction produit une position (un indice) dans le tableau représentant le filtre donc H1 donne 3 , H2 donne 4 , et H1 donne 8.

Avant insertion : toutes les cases du tableau sont initialisées à 0 tableau de taille 10.

Exemple : [0, 0, 0, 0, 0, 0, 0, 0, 0, 0]

Après insertion : les trois positions obtenues sont marquées d'un 1.

Exemple : [0, 0, 0, 1, 1, 0, 0, 0, 1, 0]

Ainsi, le *k*-mer est représenté dans le filtre par l'activation de trois bits spécifiques

3.2) La Transmission (Réduction des *k*-mers)

Afin de ne retenir que les *k*-mers suffisamment représentés pour être considérés comme fiables, une architecture de Bloom Filters en cascade a été implémentée. Cette approche permet de filtrer les erreurs de séquençage qui se manifestent généralement par des *k*-mers uniques ou de faible occurrence.

BF1 : *k*-mers vus exactement 1 fois

BF2 : *k*-mers vus exactement 2 fois

BF3 : *k*-mers vus exactement 3 fois

etc jusqu'à

BF6 : *k*-mers vus exactement 6 fois

Seuls les k-mers du filtre BF6 sont utilisés pour l'assemblage, ce qui filtre efficacement les erreurs de séquençage.

3.3) La Carrosserie (Paramètres utilisés)

3.3.1) Paramètres Généraux

Afin d'uniformiser les traitements pour l'ensemble des niveaux (0 à 10), des paramètres généraux ont été fixés :

Paramètres	Valeur
Taille du filtre	800 Mbits
Nombre de fonction de Hachage	7
Seuil de comptage	5
Taille de k-mer	30
Nombre de Seed	du Level 0 à 4 : 1 du level 5 à 10 : 50

Pour les niveaux 4-10, l'augmentation du nombre de seeds (de 10 à 100) a permis de:

- Capturer plus de composantes connexes du graphe
- Augmenter le temps de calcul de 2.5x
- Reunir plusieurs parties du génome fragmenté

3.3.2) Calcul de la taille optimale du Bloom Filter et du nombre de fonction de hachage

Afin de déterminer ses paramètres il a fallu les calculer avec les formules ci-dessous.

$$m = -\frac{n \ln p}{(\ln 2)^2} \quad k = \frac{m}{n} \ln 2$$

Figure 3.2 : *Formules mathématiques pour déterminer la taille optimale du tableau (m) et le nombre de fonctions de hachage nécessaires (k) , avec n le nombre estimé d'éléments et p la probabilité de faux positifs.*

Alors pour avoir un taux de faux positif acceptable et aussi que ça tienne dans la limite technique des ressources à disposition . Le choix de 1% de taux de faux positif a été fait pour avoir environ $m = - (10^7 \times \ln(0.01)) / (0.693)^2$, $m = - (10^7 \times (-4.605)) / 0.480$, $m \approx 95 \times 10^6$ bits ≈ 800 Mbits (valeur arrondie) soit 100 Mo, avec 6 Bloom filter chargé en même temps, ça fait 600 Mo de RAM chargé en théorie mais avec python ça sera plus.

Ensuite la classe Bloom Filter repose sur trois méthodes principales : une fonction de hachage, une méthode pour ajouter un élément, et une méthode pour vérifier sa présence dans le filtre.

La fonction de hachage utilise la librairie Hashlib avec la fonction SHA-256 qui est une fonction de hachage disponible dans cette librairie

3.4) Pneu (Extension)

L'assemblage s'effectue par extension du graphe de Bruijn à partir d'un k-mer initial. Deux fonctions principales assurent cette extension : `extension_droite()` et `extension_gauche()`.

3.4.1) Pneu droit(Extension Droite)

L'extension droite repose sur une règle fondamentale des k-mers dans un génome.

Règle fondamentale : Si l'on prend un k-mer aléatoire présent dans un génome, alors il existe un k-mer formé en enlevant le premier nucléotide et en ajoutant un des 4 nucléotides (A, T, C ou G) à la fin. Autrement dit, tout k-mer a un successeur possible dans le génome.

K-mer aléatoire A C C G C A A G T A C A G C T C A T T A

Si il y a le retrait de son premier nucléotide, il devient C C G C A A G T A C A G C T C A T T A et comme k-mer successeur, il existe entre C C G C A A G T A C A G C T C A T T A + A/T/C/G.

Le filtre de Bloom, étant une structure probabiliste, permet de déterminer si un k-mer candidat (k-mer actuel privé de sa première base, auquel on ajoute A, T, C ou G) existe ou non dans l'ensemble des k-mers fiables.

Si il y a 2 k-mer qui reviennent positif dans les choix du k-mer successeur, alors les deux branches sont continué en même temps et la plus grande branche entre les deux est choisi, cette possibilité des 2 k-mers peut être expliquer par des faux positifs ou des branches sans suite derrière, que ce soit en extension droite ou gauche.

3.4.2) Pneu Gauche (Extension Gauche)

De manière symétrique, l'extension gauche applique la règle fondamentale en sens inverse : si l'on prend un k-mer présent dans un génome, alors il existe un k-mer formé en enlevant le dernier nucléotide et en ajoutant un des 4 nucléotides au début.

Si le même pris de manière aléatoire A C C G C A A G T A C A G C T C A T T A

Si il y a le retrait de son dernier nucléotide, il devient A C C G C A A G T A C A G C T C A T T et comme k-mer successeur, il existe entre A/T/C/G + A C C G C A A G T A C A G C T C A T T .

3.5) Joint de culasse (Fusion de contigs)

3.5.1) Multi seeds.

Pour construire l'ensemble du génome, une seule seed (k-mer de départ) ne suffit pas. En effet, le graphe de Bruijn peut avoir plusieurs composantes connexes, ou l'extension peut s'arrêter prématurément. Alors il y a un certain nombre d'essais qui seront fait avec les méthodes d'extension tout en prenant à chaque fois une seed (Graine) différentes à chaque fois pour ensuite faire de la fusion des différents contigs produits.

A chaque essai, un k-mer aléatoire qui est présent au moins 6 fois c'est à dire renvoyer positif dans le Bloom Filter 6 qui est dernier dans les blooms filter en cascades

3.5.2) Fusion avec multi seeds

Lors de l'extension à partir de seeds multiples, plusieurs contigs sont générés. Ces contigs peuvent :

Problème	Illustration	Solution
Se chevaucher	Contig1: ACGTACGT... Contig2: TACGTACGT...	Fusion par chevauchement
Être inclus	Contig1: ACGTACGTACGT Contig2: CGTACGT	Garder le plus long
Être disjoints	Contig1: ACGT... Contig2: GCTA...	Conserver séparés

La fusion permet d'obtenir un assemblage plus long et plus cohérent, idéalement un seul contig représentant le génome complet.

3.6) L'influence du multi seed (Analyse Comparative)

Le multi-seed n'est pas utile pour les niveaux 0 à 4, où le nombre de contigs est de 1. À partir du niveau 5, il devient obligatoire : le nombre de contigs augmente drastiquement (48 au niveau 5, 49 au niveau 6, puis 29 au niveau 7), et le nombre d'essais par seed augmente également, permettant d'obtenir un génome plus complet. D'après la logique, à partir du niveau 5, les erreurs de séquençage créent de "fausses" branches dans le graphe de Bruijn

- Notre cascade de Bloom filters (seuil à 6) ne suffit plus à filtrer toutes les erreurs

3.7) Gestion de dodane(Bulle)

Dans un graph de Bruijn , il y a des possibilités de bulles avec 2 chemins possible avec un seul nucléotide changeant entre les 2 chemins, comme le bloom filter implémenté ne prend pas en compte le plus d'apparition dans ces bits, une méthode de choix par consensus (le chemin avec le plus de fréquence d'apparition) n'est pas possible comme il n'y a pas de possibilité de choix, l'aléatoire est choisi comme

méthode, c'est à dire que si 2 chemins ont le même score (même nombre de k-mer possible suivant) alors le choix entre les deux chemins sera aléatoire.

3.8) Voie d'amélioration

Bien que l'assembleur implémenté ait démontré son efficacité pour reconstruire des génomes à partir de données de séquençage, plusieurs améliorations pourraient être apportées pour accroître ses performances, sa précision et sa scalabilité.

3.8.1) Gestion optimisée des faux positifs

Actuellement, le filtre de Bloom utilise un taux de faux positifs de 1%. Cependant, lors des extensions, un faux positif peut conduire à une branche morte ou à un contig erroné. Pour améliorer ce point :

Il faudrait implémenter un mécanisme de vérification : lorsqu'un k-mer candidat est identifié comme présent dans BF6, une vérification supplémentaire pourrait être effectuée en consultant les filtres BF1 à BF5 pour confirmer sa fréquence réelle.

Par la suite, il faudrait ajuster dynamiquement le taux de faux positifs : pour les niveaux les plus complexes (niveaux 8-10), un taux plus faible (0,1% ou 0,01%) pourrait être utilisé, au prix d'une augmentation de la mémoire mais dû à un manque de RAM sur les ressources ce n'était pas possible.

Ou alors il faudrait peut être utiliser un filtre de Bloom partitionné , cette variante améliore la localité mémoire et réduit le nombre d'accès mémoire par requête en partitionnant la suite de bit en fonction du nombre de fonction de hachage et chaque fonction de hachage à " sa partie " . Un autre Bloom Filter possible est un counting Bloom filter

3.9) Contrôle technique (Quast)

Afin d'évaluer la qualité du génome assemblé il faut un outil comme Quast au début afin de déterminer le pourcentage de ressemblance avec le génome de référence et à partir du niveau 4, quast sera utilisé pour déterminer les différents paramètres afin de juger de la qualité du génome assemblé

3.10) Résultat de Performance (Résultat expérimental)

3.10.1) Resultat de temps et de taille de génome

Pour chaque level, la taille de k sera de 30 et chaque k-mer utilisé est vu au moins 6 fois et chaque Bloom filter est d'une taille de 1,4 milliards de bits et 7 fonctions de hachage et un seuil de contig gardé à 5 000.

level	0	1	2	3	4	5	6
temps	9 secondes	650.63 secondes	618.32 secondes	488.27 secondes	1215.17 secondes	606.62 secondes	569.57 secondes
taille du génome en pb	9935	999 960	999 976	999 956	999 979	1 426 546	1 361 076
nombre de contig	1	1	1	1	1	45	42
plus grand contig en pb (N50)	9935	999 960	999 976	999 956	999 979	122 463	70 965
nombre d'essai	1	1	1	1	1	50	50
% de génome de référence	99,35%	99.996%	99.998%	99.996%	Inconnu	Inconnu	Inconnu

Tableau 3.1 – Performances de l'assemblage pour les niveaux 0 à 6 : temps de calcul, taille du génome, nombre de contigs, N50 et nombre d'essais:

Les niveaux 1 à 4 produisent un assemblage de haute qualité avec un seul contig couvrant >99.99% du génome de référence. L'assemblage est génétiquement complet. Alors qu'à partir du niveau 5 et 6 le nombre de contig augmente et le taille du N50 diminue cela peut suggérer un surassemblage.

level	7	8	9	10
temps	869.23 secondes	1630 secondes	secondes	secondes
taille du génome en pb	4 526 850	999 979	0	0
nombre de contig	50	1	0	0

plus grand contig en pb (N50)	204584	999 979	0	0
nombre d'essai	50	50	50	50

Tableau 3.2 – Performances de l'assemblage pour les niveaux 7 à 10 : temps de calcul, taille du génome, nombre de contigs, N50 et nombre d'essais

À partir du niveau 7, l'assemblage devient catastrophique : la taille du génome est surestimée de 350% (4.5 Mpb au lieu de ~1 Mpb), suggérant une amplification de régions répétées . Les niveaux 9 et 10 n'ont produit aucun contig, probablement car le filtre de Bloom (seuil ≥ 6) n'a retenu aucun k-mer.

3.10.2) Résultat de KAT sur les niveaux 4 5 6

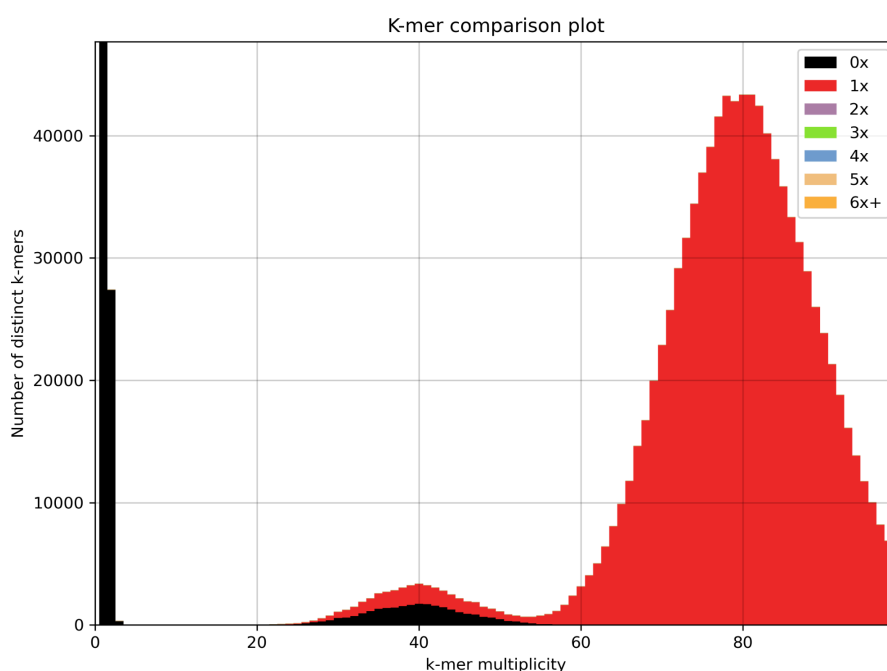


Figure 3.3 : Courbe de l'abondance des kmers distinct présent dans l'assemblage pour le niveau 4 avec un seuil de 6

Après application du seuil de multiplicité ≥ 6 , la distribution filtrée montre un pic principal situé à la multiplicité 6 (la borne inférieure du seuil), avec une décroissance rapide des k-mers pour les multiplicités supérieures.

L'absence de k-mers en dessous de 6 confirme l'élimination complète du bruit (erreurs de séquençage et régions à faible couverture).

La forme de la distribution, resserrée autour du seuil, indique que la couverture du génome est relativement uniforme et qu'il n'y a pas de régions fortement répétées (absence de queue longue).

Le fait que les k-mers conservés aient tous une multiplicité ≥ 6 , avec un pic à exactement 6, indique l'absence de régions fortement répétées (qui auraient généré des multiplicités beaucoup plus élevées). La distribution resserrée autour du seuil confirme une couverture uniforme du génome.

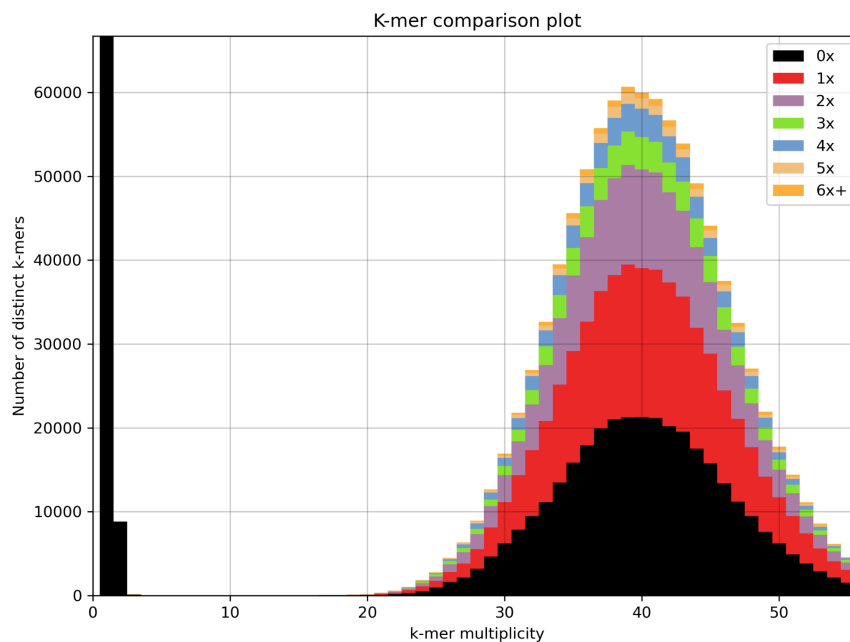


Figure 3.4 : Courbe de l'abondance des kmers distinct présent dans l'assemblage pour le niveau 5 avec un seuil de 6

Le pic à faible multiplicité ($x = 1-2$) La barre noire très haute à gauche représente les k-mers absents de l'assemblage (0x) qui apparaissent très peu de fois dans les reads ce sont typiquement des erreurs de séquençage. Cela est tout à fait normal et attendu.

Le pic principal (autour de $x \approx 38-40$) C'est ici que se concentre l'essentiel de l'information biologique : La proportion importante de 0x (noir) au pic indique effectivement que des k-mers fréquents dans les reads manquent dans l'assemblage cela peut correspondre à des régions difficiles à assembler (répétitions, forte teneur en GC).

La dominance du rouge (1x) au pic est en réalité un bon signe : la majorité des k-mers couverts sont assemblés une seule fois, donc un assemblage

majoritairement haploïde correct. La présence de 2x, 3x, 4x suggère des régions dupliquées ou répétées dans le génome assemblé, mais elles restent minoritaires

La conclusion sur les séquences répétées est partiellement correcte, mais la cause principale du 0x est davantage liée à l'effondrement de répétitions (*repeat collapse*) qu'à une simple "construction partielle"

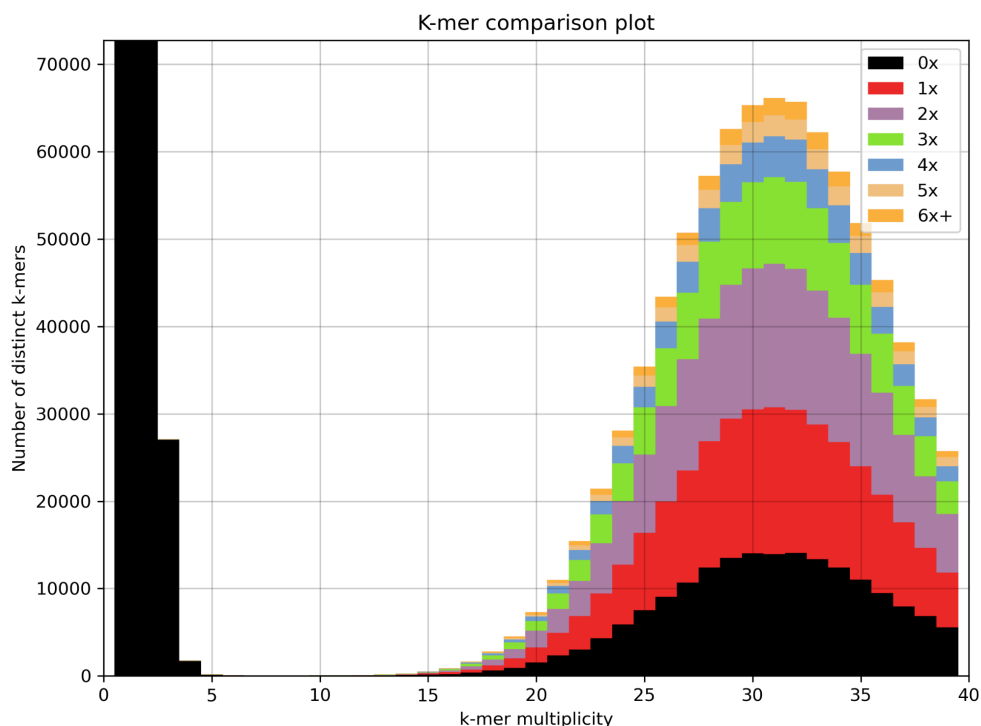


Figure 3.5 : Courbe de l'abondance des kmers distinct présent dans l'assemblage pour le niveau 6 avec un seuil de 6

Le pic principal est centré autour de $x \approx 30x$, ce qui indique une couverture de séquençage moyenne d'environ 30x. C'est légèrement inférieur au graphique précédent ($\sim 40x$).

Ce graphique indique un assemblage avec beaucoup plus de duplications que le niveau 5. Le génome assemblé contient de nombreuses séquences répétées en 3, 4, 5 copies ou plus, ce qui peut refléter soit une polyploïdie réelle du génome étudié, soit un problème d'assemblage produisant des contigs redondants.

La présence persistante de 0x suggère également que l'assemblage reste incomplet pour certaines régions.

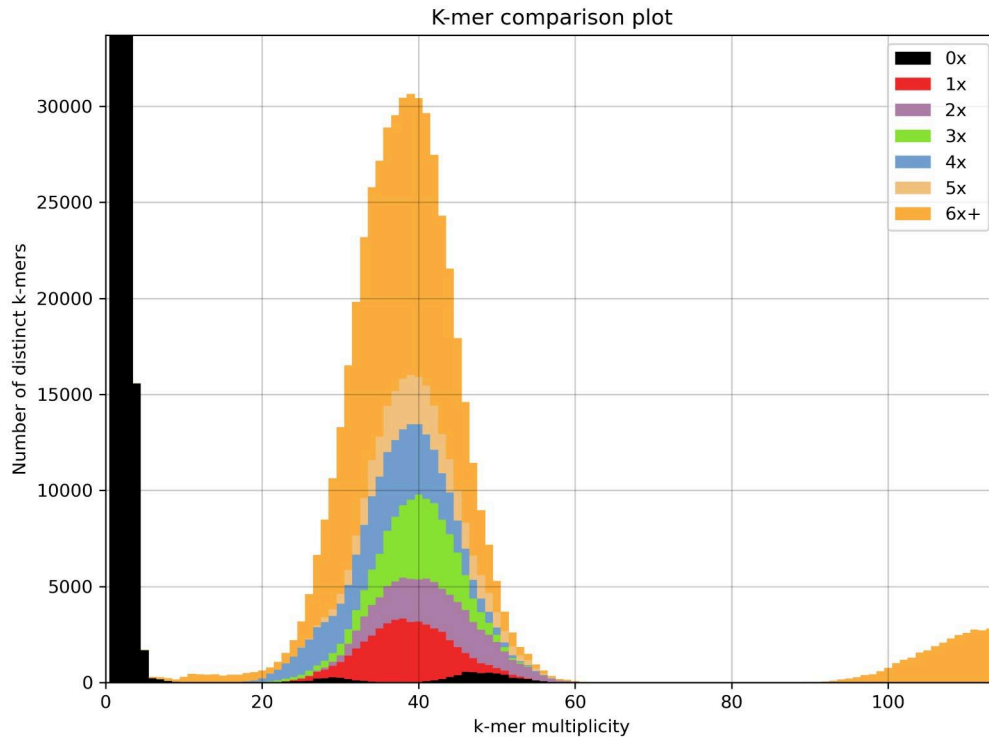


Figure 3.6 : Courbe de l'abondance des kmers distinct présent dans l'assemblage pour le niveau 7 avec un seuil de 6

Ce graphique représente le pire profil d'assemblage observé. La domination absolue du 6x+ traduit une sur-duplication massive des séquences dans l'assemblage, avec un second pic à ~105x indiquant des régions répétées en très grand nombre de copies. L'assembleur a manifestement dupliqué artificiellement la majorité des séquences, ou le génome étudié est hautement polyploïde avec de nombreux éléments répétés. Dans tous les cas, cet assemblage nécessiterait un travail important de déduplication et de purge des contigs redondants.

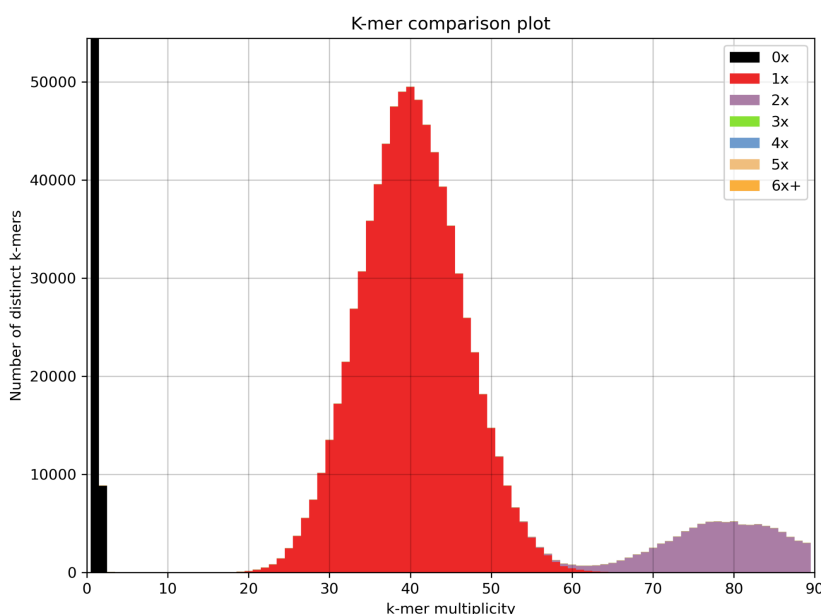


Figure 3.7 : Courbe de l'abondance des kmers distinct présent dans l'assemblage pour le niveau 8 avec un seuil de 6

Le niveau 8 représente un assemblage de qualité. La quasi-totalité des k-mers est assemblée exactement une fois (1x), avec un second pic propre à 2x correspondant aux régions dupliquées du génome. L'absence totale de copies 3x, 4x, 5x+ et la faible proportion de k-mers manquants (0x) indiquent un assemblage complet, propre, sans redondance artificielle il peut être alors un profil idéal.

4) Conclusion Comparative

Tandis que l'approche par Filtre de Bloom privilégie l'optimisation drastique de l'empreinte mémoire via une structure probabiliste, l'approche par Dictionnaire offre une précision déterministe sans risque de faux positifs. Cette dernière permet une gestion plus fine des bifurcations du graphe grâce à une exploration exhaustive des k-mers réels, au prix d'une consommation de RAM supérieure sur les génomes à forte complexité. Au final même si nos deux voitures ont des moteurs différents, en mettant les bons composants on a pu les faire rouler toutes les deux sur l'autoroute de l'assemblage par graph de Bruijn.